

Git & GitHub Collaboration Guide

This guide explains how to set up Git and GitHub, get access to the project repository, create a branch, work on your assigned task, keep your branch updated, push your changes, and submit your work for review through a Pull Request.

Important: You do not need to create your own repository for this project. The project repository belongs to the team, and you will be added as a collaborator.

1. Prerequisites

Before you start, make sure you have:

- A GitHub account
- Git installed on your computer
- Visual Studio Code (VS Code) or another code editor
- A modern web browser (Chrome, Edge, Firefox, or Safari)
- A stable internet connection

Git vs GitHub

- **Git** is the version-control software installed on your computer. It tracks changes, creates branches, and makes commits.
- **GitHub** is an online platform that hosts Git repositories and lets teams collaborate on code.

2. Create a GitHub Account

1. Go to [GitHub](https://github.com) and click **Sign up**.
2. Enter your email address, create a password, and choose a username.
3. Verify your email address and complete the account setup.

Your username appears in your profile URL, for example `https://github.com/mercy-yakubu` .

See GitHub's guide: [Getting started with your GitHub account](#).

3. Find and Share Your GitHub Profile Link

When asked for your "GitHub link", you usually need your **profile** link, not a repository link.

Method 1: From your profile

1. Log in to GitHub and click your profile picture (top-right).
2. Click **Your profile**.
3. Copy the URL from the address bar.

Method 2: Build it yourself

If your username is `johndoe` , your profile link is:

`https://github.com/johndoe`

Copy

Send this link to the team lead when requested.

4. Install Git

Windows

Download and run the installer from [Git for Windows](#). The default options are fine for most users.

macOS / Linux

Visit [Git Installation](#) and select your operating system.

Beginner video

[GitHub: A Brief Introduction to Git for Beginners](#) covers installation, configuration, basic commands, and Git vs GitHub.

5. Check That Git Is Installed

Open a terminal (Git Bash, PowerShell, Command Prompt, or VS Code's integrated terminal) and run:

```
git --version
```

Copy

You should see something like:

```
git version 2.x.x
```

Copy

The exact version number may differ. If a version is shown, Git is installed.

6. Configure Git

Before making commits, tell Git who you are.

Set your name

```
git config --global user.name "John Doe"
```

Copy

Set your email

Use an email associated with your GitHub account.

```
git config --global user.email "johndoe@gmail.com"
```

Copy

Check your configuration

Copy

```
git config --global user.name  
git config --global user.email
```

[Copy](#)

Note: Your Git username is not necessarily the same as your GitHub username. Git uses the configured name and email to associate commits with you.

Official docs: [Setting your username](#), [Setting your commit email](#), [Set up Git](#).

7. Accept the Repository Invitation

The project lead will add you as a collaborator, and GitHub will send you an invitation.

1. Check the email address connected to your GitHub account.
2. Open the invitation from GitHub (you can also check your GitHub notifications).
3. Click **Accept invitation**.

Do not create a separate repository for the project unless the team lead specifically asks you to.

8. Clone the Project Repository

Cloning downloads a copy of the repository to your computer. You only need to do it **once**.

Step 1: Get the repository link

On the project's GitHub page, click **Code** → **HTTPS** and copy the URL:

```
https://github.com/TEAM-OWNER/PROJECT-NAME.git
```

[Copy](#)

Step 2: Go to where you want the project

```
cd Desktop
```

[Copy](#)

```
cd desktop
```

Step 3: Clone it

Replace the URL with the real one:

```
git clone https://github.com/TEAM-OWNER/PROJECT-NAME.git
```

Copy

Step 4: Enter the project folder

```
cd PROJECT-NAME
```

Copy

Verify the connection

```
git remote -v
```

Copy

You should see the GitHub URL listed as `origin`.

Official docs: [Cloning a repository](#), [Getting changes from a remote](#).

9. Open the Project in VS Code

From inside the project folder, run:

```
code .
```

Copy

Or open VS Code and choose **File** → **Open Folder**, then select the cloned folder.

10. Understand Branches

A branch is a separate version of the project where you can work without changing `main`. Our project uses `main` as the default branch, and **you should not work directly on it**. Create a separate branch for each task.

branch for each task:

```
main
|
|— feature/navbar
|— feature/hero
|— feature/footer
|— feature/contact-form
```

Copy

Each person works on their own branch and submits changes through a Pull Request.

Why?

- Team members can work on different sections at the same time
- Unfinished work stays away from `main`
- Code is reviewed before it is merged
- Accidental changes to the main project are reduced
- It's easy to track who worked on each feature

Official docs: [About branches](#), [Managing branches](#).

11. Create Your Feature Branch

First make sure your local `main` is up to date.

Step 1: Switch to main

```
git switch main
```

Copy

Step 2: Get the latest main

```
git pull origin main
```

Copy

Step 3: Create your branch

```
git switch -c feature/navbar
```

Copy

This creates the branch **and** switches you to it.

Check your current branch

```
git branch
```

Copy

The current branch has an asterisk:

```
* feature/navbar  
main
```

Copy

12. Work on Your Assigned Task

If you were assigned the Navbar (`feature/navbar`), you can work on the navbar HTML, CSS, JavaScript, and any other files specifically related to your task.

Important: Do not modify another person's section unnecessarily. If you notice something outside your task that needs attention, tell the team lead or create an issue/comment so the team can decide how to handle it.

13. Check Your Changes

Before committing, see what Git thinks has changed:

```
git status
```

Copy

It shows modified, new, and deleted files, plus your current branch. To review the exact changes:

```
git diff
```

Copy

Test your work

- Test the feature in your browser
- Check desktop and mobile layouts
- Check for console errors
- Make sure your assigned functionality works
- Make sure you haven't broken another section

14. Commit Your Changes

A commit is a saved checkpoint of your changes.

Step 1: Stage your changes

```
git add .
```

Copy

Step 2: Create the commit

```
git commit -m "Add navbar section"
```

Copy

Good commit messages

```
Add navbar section
Add responsive hero styles
Fix mobile navigation
Add contact form validation
Update footer layout
```

Copy

Avoid vague messages

```
stuff
changes
```

Copy

update
fix
done

15. Push Your Branch to GitHub

The first time you push a new branch:

```
git push -u origin feature/navbar
```

Copy

Replace `feature/navbar` with your actual branch name. After that, this is enough:

```
git push
```

Copy

Your branch will now appear in the project repository on GitHub.

16. Create a Pull Request

A Pull Request (PR) asks to merge your branch into another branch, usually `main`. Do not merge directly into `main` unless the team lead asks you to.

1. Go to the project repository on GitHub.
2. Open the **Pull requests** tab and click **New pull request**.
3. Select your branch as the source and `main` as the target.
4. Review the changes shown by GitHub.
5. Give the PR a clear title and describe what you changed.
6. Mention anything the reviewer should know.
7. Request a review from the appropriate team member or team lead.
8. Click **Create pull request**.

Example PR title

Add responsive navbar

Copy

Example PR description

Copy

Changes

- Added the navbar HTML structure
- Added responsive navbar styles
- Added mobile menu interaction

Testing

- Tested on desktop
- Tested on mobile
- Checked browser console for errors

Notes

- Ready for review

Official docs: [Creating a Pull Request](#), [Hello World](#). Video: [How to Create a Pull Request](#).

17. Code Review

A reviewer (a team member or the team lead) can comment on specific lines, ask questions, suggest improvements, request changes, or approve the PR.

If changes are requested

Do **not** create another Pull Request. Make the changes on the same branch:

Copy

```
git add .
git commit -m "Address review comments"
git push
```

The existing PR updates automatically. Once the reviewer is satisfied, it can be approved and merged.

18. Keep Your Local Project Updated

18. Keep Your Local Project Updated

Teammates keep adding changes to `main`, so your local copy can become outdated. Update it regularly.

Update main

```
git switch main
git pull origin main
```

Copy

Update an existing feature branch

To bring the latest `main` into the branch you're working on:

```
git switch main
git pull origin main
git switch feature/navbar
git merge main
```

Copy

Replace `feature/navbar` with your actual branch.

Important: Do not copy files from another person's branch into yours. Use Git to bring changes in.

Why do this?

- You work with the latest version of the project
- Conflicts are detected earlier
- You avoid large conflicts near the end of your task
- Your code is checked against the team's latest code

19. After Your Pull Request Is Merged

Once your PR is merged into `main`, your feature is part of the project. Before starting another task:

```
git switch main
git pull origin main
git switch -c feature/employee-card
```

[Copy](#)

Do not keep working on an already-merged branch. Create a new branch for each task.

20. Recommended Branch Naming

Use clear, consistent names.

Features

```
feature/navbar
feature/hero-section
feature/about-section
feature/contact-form
feature/footer
```

[Copy](#)

Bug fixes

```
fix/mobile-navbar
fix/form-validation
fix/responsive-layout
```

[Copy](#)

Documentation

```
docs/update-readme
docs/git-guide
```

[Copy](#)

Examples

```
# Assigned the Hero section
git switch -c feature/hero-section
```

[Copy](#)

```
# Fixing the mobile navbar  
git switch -c fix/mobile-navbar
```

21. Quick Git Workflow

Start a new task

```
git switch main  
git pull origin main  
git switch -c feature/your-task
```

Copy

Work, check, stage, commit

Make your changes in VS Code, then:

```
git status  
git add .  
git commit -m "Describe your changes"
```

Copy

Push

```
# First push  
git push -u origin feature/your-task  
  
# After that  
git push
```

Copy

Create the Pull Request

On GitHub: **Pull Requests** → **New Pull Request**, then `your-branch` → `main` .

After review (if changes are requested)

```
git add .  
git commit -m "Address review comments"  
git push
```

[Copy](#)

After merge

```
git switch main  
git pull origin main
```

[Copy](#)

Then create a new branch for your next task.

22. Common Git Commands

Command	What it does
<code>git --version</code>	Checks whether Git is installed
<code>git status</code>	Shows the current state of your working directory
<code>git branch</code>	Shows your local branches
<code>git switch main</code>	Switches to the <code>main</code> branch
<code>git switch -c branch-name</code>	Creates and switches to a new branch
<code>git add .</code>	Stages all changed files
<code>git commit -m "message"</code>	Saves staged changes as a commit
<code>git push</code>	Uploads your commits to GitHub
<code>git pull origin main</code>	Downloads and integrates the latest <code>main</code> changes
<code>git clone URL</code>	Downloads a repository from GitHub
<code>git diff</code>	Shows changes that have not been committed
<code>git remote -v</code>	Shows the remote repository connected to your project

23. Common Problems

"git is not recognized"

Git is probably not installed correctly, or your terminal can't find it. Check:

```
git --version
```

Copy

If that fails, reinstall Git from git-scm.com, then restart your terminal or VS Code.

"I cannot clone the repository"

1. Did you accept the GitHub repository invitation?
2. Are you logged into the correct GitHub account?
3. Did you copy the correct repository URL?
4. Do you have permission to access the repository?
5. Is your internet connection working?

"I pushed my branch but cannot see it on GitHub"

```
git push -u origin your-branch-name
```

Copy

Then refresh the repository page on GitHub.

"I don't know which branch I'm on"

```
git branch  
git status
```

Copy

The branch marked with `*` is your current branch.

"I accidentally made changes on main"

Stop and tell the team lead before pushing. Do not push those changes to main. The team

Stop and tell the team lead **before pushing**. Do not push those changes to `main`. The team lead can help you move them to the correct feature branch.

"I have a merge conflict"

A merge conflict happens when Git cannot automatically combine changes from different branches. Do not randomly delete code to make the error disappear.

1. Don't panic.
2. Read the files Git identifies as conflicted.
3. Ask the team lead or another team member for help.
4. Resolve the conflict carefully.
5. Test the project after resolving it.
6. Commit the resolution.

24. Useful Resources

Git

- [Official Git Website](#)
- [Git Installation](#)
- [Pro Git Book](#)

GitHub

- [GitHub](#) and [GitHub Documentation](#)
- [Getting Started with GitHub](#)
- [Set Up Git](#)
- [GitHub Hello World](#)
- [Working with Branches](#)
- [Creating Pull Requests](#)
- [Getting Changes from a Remote Repository](#)

Beginner videos

- [A Brief Introduction to Git for Beginners](#)

- [How to Create a Pull Request](#)

The Workflow in One Diagram



Copy

Golden Rules

1. Do not work directly on `main`.
2. Create a separate branch for your task.
3. Pull the latest `main` before starting a new task.
4. Test your code before pushing.
5. Write clear commit messages.
6. Push your branch and create a Pull Request.
7. Do not merge your own Pull Request unless the team workflow specifically allows it.
8. If you are confused or encounter an error, ask before making random changes.

